시계열 데이터 분석 실습 가이드

국내 주식 시세로 배우는 이동평균·추세 분석

> *이 자료는 공공데이터(미세먼지·기상청, KOSIS) 실습에 이어, 시간 순서가 핵심인 "시계열 데이터"를 다루는 실습 사례입니다. 삼성전자(005930) 일별 시세로 ①이동평균선과 추세 전환 신호(골든크로스·데드크로스, 기초)를 직접 계산·시각화하고, ②Prophet 통계 모델로 미래 구간을 예측하고 백테스트로 검증(중급)까지 다룹니다. 실제 주차 배치는 추후 별도로 결정합니다.*

1. 이 실습의 목적

지금까지의 실습(에어코리아·기상청, KOSIS)은 "특정 시점의 지역별 비교"가 중심이었다면, 이번 실습은 "시간이 흐르며 하나의 값이 어떻게 변하는가"를 다루는 시계열(time series) 데이터입니다. 같은 숫자 데이터라도 분석 관점이 "지역 간 비교"에서 "과거-현재 흐름과 추세"로 바뀐다는 것을 체험하는 것이 핵심 목표입니다.

국내 주식 시세를 API 로 수집하여 이동평균선을 계산하고, 이동평균선이 서로 교차하는 지점(골든크로스·데드크로스)을 코드로 탐지·시각화하여 추세 전환을 해석해봅니다.

이 실습에서 얻어가는 핵심 교훈

- 데이터 출처가 예고 없이 바뀔 수 있다 — 크롤링은 사이트 구조 변경에 취약하며, 목적에 맞는 전용 라이브러리(pykrx)가 더 안정적일 수 있다.
- boolean 값의 diff() 연산은 -1/0/1 로 나오므로, "==True/False"만으로 비교하면 원하는 조건을 놓칠 수 있다 — 조건을 정확한 숫자로 비교해야 한다.
- 기술적 지표(이동평균 교차)는 만능이 아니다 — 뚜렷한 추세 구간에서는 신호가 유효하지만, 橫보(박스권) 구간에서는 잦은 거짓 신호(휩쏘)를 낸다.
- 단일 지표·신호만 보지 말고 반드시 시각화로 확인해야 한다 — 미세먼지·인구·고용 실습에서도 반복된, 이 과정 전체를 관통하는 교훈이다.

2. 실습 개요

항목	내용
대상 종목	삼성전자(005930)
데이터 기간	2025-01-01 ~ 2026-09-23 (421 거래일)
기법 수준	기초 — 이동평균선(5·20·60 일), 골든크로스/데드크로스 / 중급 — Prophet 통계적 시계열 예측
도구	Python(pandas, matplotlib), pykrx, prophet · Google Colab

> *예측(prediction)이라는 표현을 썼지만, 이동평균·추세 분석은 엄밀히 말하면 "과거 데이터로 현재 추세를 해석하는 기법"이지 미래를 예측하는 통계·머신러닝 모델은 아닙니다. Prophet·ARIMA·LSTM 같은 본격적인 시계열 예측 모델은 이 실습의 다음 단계로 이어질 수 있습니다.*

3. 1 단계 — 데이터 수집

처음에는 국가통계포털 실습과의 일관성을 위해 네이버금융(finance.naver.com)의 "종목별 시세" 페이지를 크롤링하는 방식으로 시작했습니다.

```
# 최초 시도 — 네이버금융 크롤링
url = f"https://finance.naver.com/item/sise_day.naver?code={code}&page=1"
res = requests.get(url, headers={"User-Agent": "Mozilla/5.0"})
tables = pd.read_html(res.text)
```

> *트러블슈팅: 위 코드를 실행하자 표 대신 "이 페이지는 더 이상 제공되지 않습니다. Npay 증권 바로가기" 라는 안내 문구만 반환되었습니다. 네이버금융이 기존 "종목별 시세" 페이지를 신규 도메인(Npay 증권, m.stock.naver.com)으로 이전하면서 옛 URL 이 더 이상 데이터를 주지 않게 된 것입니다. 크롤링은 사이트 구조가 바뀌는 순간 예고 없이 깨진다는 것을 보여주는 실제 사례입니다.*

이 문제를 해결하는 방법은 새 페이지 구조를 다시 분석해 크롤링 코드를 고치거나, 한국거래소(KRX) 데이터 전용 라이브러리로 바꾸는 것입니다. 사이트 구조 변경에 흔들리지 않는 후자를 선택해 pykrx 로 전환했습니다.

```
!pip install pykrx -q

from pykrx import stock

code = "005930"  # 삼성전자
df = stock.get_market_ohlcv("20250101", "20260923", code)
df = df.sort_index()
```

> *실행 시 "KRX 로그인 실패" 메시지가 함께 출력되지만 무시해도 됩니다 — 이는 공매도 잔고 등 일부 고급 기능에만 필요한 선택적 로그인이며, 일별 시세(OHLCV) 조회에는 영향이 없습니다.*

pykrx 로 받은 데이터는 시가·고가·저가·종가·거래량·등락률까지 이미 숫자형으로 정리되어 있고 날짜가 인덱스로 잡혀 있어, 크롤링과 비교해 별도 정제 과정이 거의 필요 없었습니다(421 행 × 6 열).

4. 2 단계 — 이동평균선 계산 및 시각화

```
df["MA5"] = df["종가"].rolling(window=5).mean()
df["MA20"] = df["종가"].rolling(window=20).mean()
df["MA60"] = df["종가"].rolling(window=60).mean()
```

5 일선은 단기, 20 일선은 중기, 60 일선은 장기 추세를 나타냅니다. 세 이동평균선과 실제 종가를 함께 그리면 아래와 같은 결과를 얻습니다.

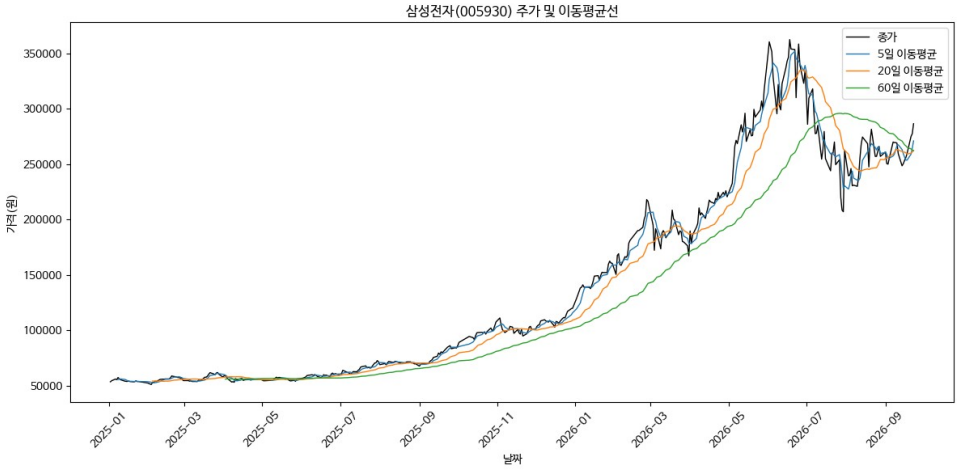


그림 1. 삼성전자 종가 및 5·20·60 일 이동평균선 (2025-01~2026-09)

2025 년 초 5 만원대였던 주가가 2026 년 6~7 월 36 만원대까지 급등했다가, 이후 25 만원대로 조정을 받고 다시 반등하는 흐름이 뚜렷하게 나타납니다.

5. 3 단계 — 추세 분석: 골든크로스와 데드크로스

골든크로스는 단기 이동평균선(MA5)이 장기 이동평균선(MA20)을 아래에서 위로 뚫고 올라가는 시점으로 상승 추세 전환 신호로, 데드크로스는 그 반대로 하락 전환 신호로 해석됩니다.

5.1 트러블슈팅 — boolean diff() 비교 오류

처음에는 아래와 같이 작성했습니다.

```
df["MA5_above_MA20"] = df["MA5"] > df["MA20"]
df["cross"] = df["MA5_above_MA20"].diff()
golden_cross = df[df["cross"] == True]
dead_cross = df[df["cross"] == False]   # 오류: 변화 없음(0)을 걸러낼 뿐, 하락 돌파(-1)가 아님
```

> *boolean 컬럼을 diff() 하면 상승 돌파는 1.0, 하락 돌파는 -1.0, 변화 없음은 0.0 으로 계산됩니다. "== True" 는 1.0 을 정확히 걸러내 골든크로스 탐지는 문제가 없었지만, "== False" 는 -1.0 이 아니라 0.0(변화 없음)을 걸러내는 조건이라, "데드크로스" 결과가 실제로는 "변화가 없었던 날들" 이 우연히 표시된 것이었습니다. 조건을 -1/1 로 명확히 비교하도록 수정했습니다.*

```
df["cross"] = df["MA5_above_MA20"].astype(int).diff()
golden_cross = df[df["cross"] == 1]
dead_cross = df[df["cross"] == -1]
```

5.2 결과 해석

수정 후 교차 시점을 실제 가격 흐름과 맞춰보면, 2026-07-02 데드크로스(28 만 6 천원)가 가장 뚜렷합니다 — 직전 6 월 말 36 만원 근처 최고점을 찍은 직후로, 이 데드크로스 이후 주가는 20 만원대까지 큰 폭 조정을 받았습니다. 가장 최근인 2026-09-22 골든크로스(27 만 7 천 5 백원)는 최근 반등(25 만원대→ 28 만 6 천 5 백원)과 정확히 맞물립니다.

반면 8~9 월 사이에는 골든크로스·데드크로스가 짧은 간격으로 5 차례나 반복되었습니다 — 뚜렷한 추세 없이 박스권에서 등락한 시기였다는 뜻으로, 이 구간에서 매번 신호를 따라 매매했다면 손실을 봤을 가능성이 큽니다.

6. 4 단계 — 교차 시점 시각화

골든크로스·데드크로스 발생 지점을 차트 위에 마커로 직접 표시하면, 신호가 유효했던 구간과 그렇지 않았던 구간이 한눈에 대비됩니다.

```
plt.scatter(golden_cross.index, golden_cross["종가"], marker="^", color="red", s=120,
zorder=5, label="골든크로스")
plt.scatter(dead_cross.index, dead_cross["종가"], marker="v", color="blue", s=120,
zorder=5, label="데드크로스")
```

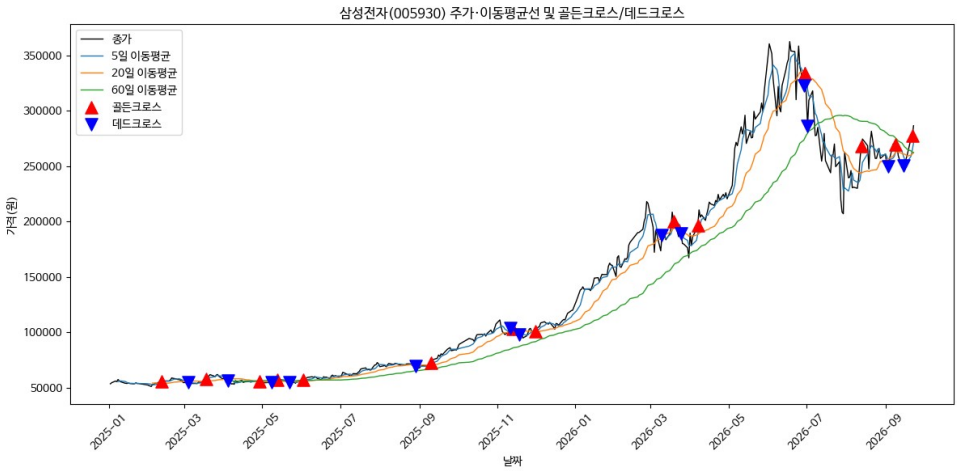


그림 2. 주가·이동평균선 위에 표시한 골든크로스(▲)·데드크로스(▼)

2025 년 2~6 월 횡보 구간과 2026 년 8~9 월 박스권 구간에는 빨간·파란 마커가 촘촘하게 몰려 있어 "휩쏘"(잦은 거짓 신호)를 시각적으로 확인할 수 있습니다. 반대로 2025 년 11 월부터 2026 년 6 월까지 이어진 뚜렷한 상승 추세 구간에는 골든크로스 이후 한참 동안 데드크로스가 나오지 않아, 신호가 적고 신뢰도가 높다는 것도 함께 드러납니다.

7. 종합 교훈

이동평균 교차 기법은 "추세장에서는 유효하지만 횡보장에서는 잦은 거짓 신호를 낸다"는 뚜렷한 한계를 가지고 있습니다. 단일 신호·지표만 보고 판단하면 안 되고, 반드시 시각화로 그 신호가 어떤 국면에서 유효했는지 확인해야 한다는 것은 이 과정의 다른 실습(미세먼지의 상한효과, 인구의 구조적 격차, 고용·실업률의 통계적 맹점)에서도 반복적으로 확인된 교훈입니다.

> *심화 방향: 이번 실습은 "기초 — 이동평균·추세선" 수준이었습니다. 다음 단계로는 Prophet·ARIMA 같은 통계적 시계열 예측 모델로 미래 주가 구간을 추정해보거나, LSTM 딥러닝 모델로 더 정교한 예측을 시도해볼 수 있습니다.*

8. 트러블슈팅 종합표

증상	원인	해결
네이버금융 페이지가 "더 이상 제공되지 않습니다" 안내만 반환	구 도메인 페이지가 Npay 증권으로 이전	크롤링 대신 pykrx 라이브러리로 전환
"KRX 로그인 실패" 경고 메시지	공매도 잔고 등 일부 고급 기능용 선택적 로그인 안내	일별 시세(OHLCV) 조회에는 영향 없어 무시 가능
데드크로스 결과가 실제와 다름	boolean.diff() 값(-1/0/1)을 "==False"로 비교 — 0(변화 없음)을 걸러냄	"==-1"로 명확히 비교하도록 조건 수정
새 노트북에서 NameError(pd·df 없음)	커널이 초기화되어 이전 셀의 변수·임포트가 모두 사라짐	설치·임포트·수집·계산을 한 셀에 통합해 재실행
Prophet 예측값이 토·일요일마다 이상치로 떨어짐	make_future_dataframe()이 기본값으로 주말 포함 전체 달력일을 생성	freq="B"(영업일)로 미래 구간 생성
"Yearly seasonality...unstable" 경고 및 연간 패널의 부자연스러운 큰 봉우리	21 개월치(2 년 미만) 데이터로 연간 계절성 추정이 불안정 — 일회성 사건을 계절 패턴으로 오인	짧은 데이터 기간에서는 연간 계절성 해석에 주의하거나 비활성화 고려

9. 부록 — 통합 코드

```
!pip install pykrx -q
!apt-get -qq -y install fonts-nanum > /dev/null

import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
from pykrx import stock

fontpath = "/usr/share/fonts/truetype/nanum/NanumGothic.ttf"
fm.fontManager.addfont(fontpath)
plt.rcParams['font.family'] = fm.FontProperties(fname=fontpath).get_name()
plt.rcParams['axes.unicode_minus'] = False

code = "005930"  # 삼성전자
df = stock.get_market_ohlcv("20250101", "20260923", code).sort_index()

df["MA5"] = df["종가"].rolling(window=5).mean()
df["MA20"] = df["종가"].rolling(window=20).mean()
df["MA60"] = df["종가"].rolling(window=60).mean()

df["MA5_above_MA20"] = df["MA5"] > df["MA20"]
df["cross"] = df["MA5_above_MA20"].astype(int).diff()
golden_cross = df[df["cross"] == 1]
dead_cross = df[df["cross"] == -1]
```

함수	설명
stock.get_market_ohlcv(시작일, 종료일, 종목코드)	일별 시가·고가·저가·종가·거래량·등락률 조회
stock.get_market_cap(...)	일별 시가총액 조회 (이번 실습에서는 미사용)
stock.get_index_ohlcv(...)	코스피·코스닥 등 지수 시세 조회 (이번 실습에서는 미사용)

10. 심화 실습 — 통계적 시계열 예측(Prophet)

지금까지의 이동평균·교차 분석은 “과거 흐름을 요약해 현재 추세를 해석” 하는 기법이었습니다. 이번에는 한 단계 더 나아가, 메타(Meta)가 만든 시계열 예측 라이브러리 Prophet 으로 미래 주가 구간을 통계적으로 추정해봅니다. ARIMA 도 같은 계열의 기법이지만, Prophet 은 코드가 짧고 결과가 시각적으로 뚜렷해 실습용으로 선택했습니다.

10.1 데이터 형식 변환

Prophet 은 입력 데이터프레임의 컬럼명이 반드시 날짜 컬럼 “ds”, 예측 대상 값 컬럼 “y”여야 합니다. 앞서 만든 삼성전자 시세 데이터를 그대로 재사용해 형식만 변환했습니다.

```
prophet_df = df.reset_index()[["날짜", "종가"]].rename(columns={"날짜": "ds", "종가": "y"})
```

10.2 모델 학습과 예측 — 트러블슈팅: 주말 이상치

```
from prophet import Prophet

model = Prophet(daily_seasonality=False, weekly_seasonality=True, yearly_seasonality=True)
model.fit(prophet_df)

future = model.make_future_dataframe(periods=30)  # 최초 시도
forecast = model.predict(future)
```

트러블슈팅: 최초 예측 결과에서 특정 날짜의 예측값(yhat)이 갑자기 크게 떨어지는 이상치가 반복적으로 나타났습니다. 날짜를 확인해보니 전부 토요일·일요일이었습니다. 원인은 make_future_dataframe()이 기본값으로 달력상의 모든 날(주말 포함)을 만들기 때문입니다 — 정작 학습 데이터는 주식 거래일(평일)만 있어 모델이 "주말"이라는 요일 패턴을 제대로 학습한 적이 없는데, 그 상태로 주말을 예측하려니 왜곡된 값이 나온 것입니다. freq="B"(영업일)로 미래 구간을 만들도록 수정해 해결했습니다.

```
future = model.make_future_dataframe(periods=30, freq="B")  # B = 영업일(월~금)
forecast = model.predict(future)
```

모델 학습 시 "Yearly seasonality is enabled with less than 730 days(약 2 년)..."라는 경고가 함께 출력되었습니다. 저희 데이터는 21 개월치뿐이라 연간 계절성 추정이 불안정할 수 있다는 경고이며, 이 경고는 뒤의 10.3 에서 실제로 확인됩니다.

10.3 결과 시각화

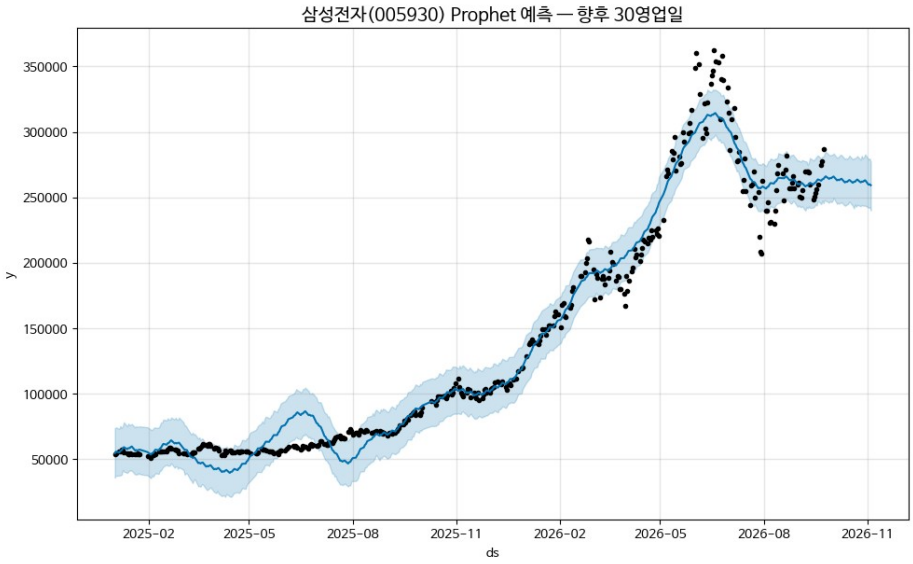


그림 3. Prophet 예측 — 실제값(검은 점)과 예측선·신뢰구간(파란 음영)

6~7 월 급등 구간에서는 실제값이 신뢰구간 상단을 뚫고 튀어나가는 지점들이 보입니다 — 그만큼 그 시기의 급등이 모델이 예상한 정상 변동 범위를 벗어난 이례적 사건이었다는 뜻입니다.

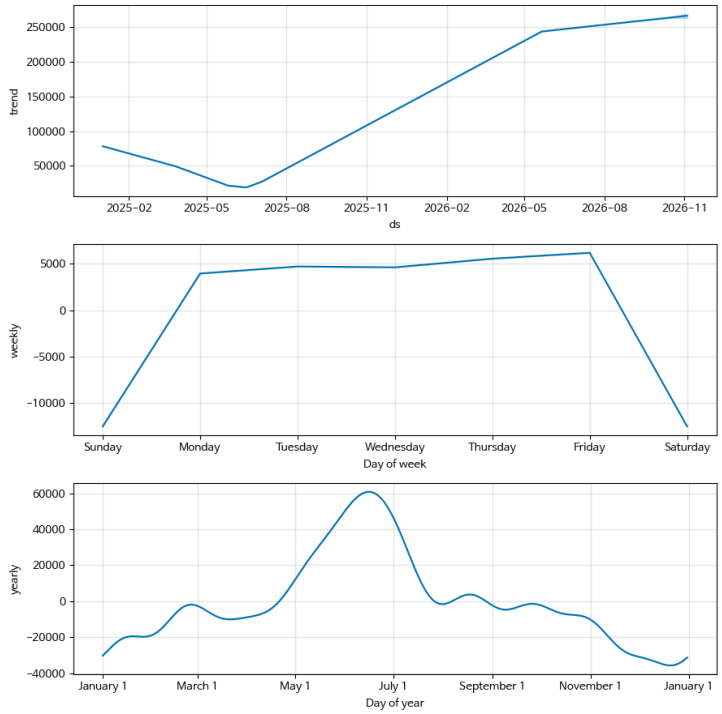


그림 4. 구성요소 분해 — 추세(trend)·요일별(weekly)·연간(yearly) 패턴

> *구성요소 분해 그래프 해석 시 주의점 두 가지: ①weekly 패널은 토·일요일에 급격히 낮게 나오지만, 이는 모델이 내부적으로 학습한 요일 함수를 7 일 전부에 대해 그린 것일 뿐 freq="B"로 실제 예측에는 영향이 없습니다. ②yearly 패널의 6~7 월 큰 봉우리는 10.2 의 경고가 실제로 나타난 사례로, 21 개월치 데이터로는 "매년 반복되는 계절 패턴"과 "올해 한 번 있었던 급등·조정"을 모델이 구분하지 못합니다 — 진짜 계절성이 아니라 일회성 사건을 계절 패턴으로 잘못 해석했을 가능성이 큽니다.*

10.4 백테스트 — 예측 성능 검증

최근 30 거래일을 떼어놓고(홀드아웃), 그 이전 391 일치 데이터만으로 모델을 다시 학습시켜 이 구간을 예측한 뒤 실제값과 비교했습니다.

```
train_df = prophet_df.iloc[:-30]
test_df = prophet_df.iloc[-30:]

model2 = Prophet(daily_seasonality=False, weekly_seasonality=True,
yearly_seasonality=True)
model2.fit(train_df)
future2 = model2.make_future_dataframe(periods=30, freq="B")
forecast2 = model2.predict(future2)

compare = forecast2[["ds", "yhat"]].merge(test_df, on="ds", how="inner")
mae = (compare["y"] - compare["yhat"]).abs().mean()
mape = ((compare["y"] - compare["yhat"]).abs() / compare["y"]).mean() * 100
```

날짜	실제 종가	예측값(yhat)	오차율(%)
2026-08-12	255,500	269,662	-5.54
2026-08-19	247,500	276,150	-11.58
2026-08-25	257,000	278,400	-8.33
2026-09-02	250,500	278,227	-11.07
2026-09-11	259,500	279,787	-7.82
2026-09-14	248,500	278,584	-12.11
2026-09-18	260,000	282,937	-8.82

2026-09-22	277,500	283,880	-2.30

평가 대상 29 거래일 기준 MAE(평균절대오차) 16,484 원, MAPE(평균절대오차율) 6.41%였습니다. 숫자 자체보다 더 중요한 것은 오차의 방향입니다 — 대부분의 날짜에서 예측값이 실제값보다 높게 나왔습니다(오차율이 거의 전부 음수). 학습 데이터가 6~7 월 폭등이 반영된 구간까지 포함하고 있어, 모델이 "이 상승 추세가 계속될 것"이라 보고 계속 27 만~28 만원대를 예측했지만, 실제로는 그 이후 25 만원대로 조정이 진행 중이었기 때문입니다.

> *핵심 통찰: 이동평균 교차 분석(5 장)과 Prophet 백테스트(10.4)는 기법은 전혀 다르지만 똑같은 한계에 도달했습니다 — 골든크로스·데드크로스도 "이미 추세가 바뀐 뒤에야" 신호가 나오는 후행 지표였고, Prophet 도 "추세가 막 꺾인 직후에는 과거 추세를 계속 따라가려 하며 방향 전환을 놓친다"는 같은 성격의 약점을 보였습니다. 기법을 바꿔도 "추세 반전 시점 자체를 예측하기는 어렵다"는 시계열 분석의 근본적인 한계는 남는다는 것을 두 가지 서로 다른 방법으로 확인한 셈입니다.*